

LAPORAN PRAKTIKUM 1

IOT II



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

LAPORAN PRAKTIKUM: DIGITAL OUTPUT ESP32 (LED BLINK)

1. Pendahuluan & Ringkasan Sistem

Praktikum ini bertujuan untuk mengimplementasikan kendali keluaran digital (*digital output*) menggunakan mikrokontroler ESP32. Sistem dikembangkan pada platform simulasi Wokwi untuk mengontrol sebuah LED berwarna merah agar berkedip secara periodik dengan interval waktu 1 detik. Selain itu, sistem dikonfigurasi untuk mengirimkan log status operasional secara real-time ke Serial Monitor.

2. Desain Rangkaian Hardware

Tabel Konfigurasi Komponen

Komponen	Pin / Terminal	Fungsi Utama
ESP32 Board	GPIO 5, GND, TX/RX	Unit pemroses utama dan sumber sinyal kendali.
Resistor 220	Pin 1 ke GPIO 5 Pin 2 ke Anoda LED	Membatasi arus listrik (<i>current limiter</i>) agar LED tidak terbakar.
LED Merah	Anoda (+) ke Resistor Katoda (-) ke GND	Indikator keluaran visual (menyala / padam).
Serial Monitor	TX ke RX, RX ke TX	Jalur komunikasi data teks antara ESP32 dan komputer.

Alur Sirkuit Terintegrasi

- Sinyal keluaran dikirim dari pin GPIO 5 ESP32 menuju terminal pertama Resistor 220 Ω .
- Arus listrik melewati resistor dan diteruskan ke terminal Anoda (positif) LED.
- Arus keluar melalui terminal Katoda (negatif) LED menuju pin GND (Ground) ESP32 untuk menutup loop sirkuit.

3. Analisis Kode Program (Software)

Kode program ditulis menggunakan bahasa C/C++ pada framework Arduino. Program dibagi menjadi tiga bagian utama: deklarasi pin, konfigurasi awal (*setup*), dan perulangan utama (*loop*).

A. Deklarasi Variabel

C++

```
const int ledPin = 5;
```

Variabel ledPin dideklarasikan sebagai konstanta bertipe integer dengan nilai 5. Nilai ini merepresentasikan pin fisik GPIO 5 pada papan ESP32 yang tersambung ke LED.

B. Fungsi Inisialisasi (setup)

C++

```
void setup() {  
  Serial.begin(115200);  
  pinMode(ledPin, OUTPUT);  
  Serial.println("Praktikum 1 - Digital Output Dimulai!");  
}
```

- `Serial.begin(115200)`: Mengaktifkan jalur komunikasi serial asynchronous pada kecepatan transfer data 115200 baud.
- `pinMode(ledPin, OUTPUT)`: Konfigurasi tingkat rendah untuk menentukan bahwa pin GPIO 5 berfungsi sebagai jalur keluaran tegangan (*output*), bukan pembaca sinyal.
- `Serial.println(...)`: Mencetak pesan pembuka ke Serial Monitor saat mikrokontroler pertama kali dinyalakan.

C. Fungsi Perulangan Utama (loop)

C++

```
void loop() {  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED Menyala");  
    delay(1000);  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED Mati");  
    delay(1000);  
}
```

Fungsi ini dieksekusi berulang kali selama mikrokontroler menerima daya:

1. Kondisi LED Menyala:

- `digitalWrite(ledPin, HIGH)` memberikan tegangan logika HIGH (sekitar 3.3 Volt) pada pin GPIO 5, menyebabkan arus mengalir dan LED menyala.
- Sistem mengirimkan string "LED Menyala" ke Serial Monitor.
- `delay(1000)` menghentikan eksekusi perintah berikutnya selama 1.000 milidetik (1 detik), mempertahankan kondisi LED tetap menyala.

2. Kondisi LED Padam:

- `digitalWrite(ledPin, LOW)` mengubah tegangan pin GPIO 5 menjadi LOW (0 Volt), menghentikan aliran arus sehingga LED padam.
- Sistem mengirimkan string "LED Mati" ke Serial Monitor.
- `delay(1000)` menahan posisi ini selama 1 detik sebelum siklus kembali ke baris pertama fungsi `loop()`.

4. Kesimpulan

Sistem berhasil mengimplementasikan prinsip dasar keluaran digital. Dengan memanfaatkan fungsi `digitalWrite()` dan pengaturan pewaktuan `delay()`, ESP32 mampu mengontrol komponen eksternal (LED) serta melakukan monitoring status sistem secara langsung melalui komunikasi Serial.

Simulasi

00:27.616 98%



LED Mati

LED Menyala

LED Mati

LED Menyala

LED Mati

LED Menyala

LED Mati

Graph, Pause, Delete, and Dropdown icons

This screenshot shows a simulation of an ESP32 microcontroller connected to an LED. The LED is currently off. The console log shows a sequence of alternating 'LED Mati' and 'LED Menyala' messages. The simulation time is 00:27.616 and the progress is at 98%.

Simulasi

00:41.172 104%



LED Menyala

LED Mati

LED Menyala

LED Mati

LED Menyala

LED Mati

LED Menyala

Graph, Pause, Delete, and Dropdown icons

This screenshot shows the same simulation, but the LED is now on. The console log shows a sequence of alternating 'LED Menyala' and 'LED Mati' messages. The simulation time is 00:41.172 and the progress is at 104%.

sketsa.ino

diagram.json

Manajer Perpustakaan ▼

```
1  konstanta int ledPin = 5 ;
2
3  void setup ( ) {
4      Serial.begin ( 115200 ) ;
5      pinMode ( ledPin, OUTPUT ) ;
6      Serial . println ( "Praktikum 1 - Output Digital Dimulai!" ) ;
7  }
8
9  void loop ( ) {
10     Tulis digital ( ledPin, TINGGI ) ;
11     Serial . println ( "LED Menyala" ) ;
12     tunda ( 1000 ) ;
13
14     Tulis digital ( ledPin, RENDAH ) ;
15     Serial . println ( "LED Mati" ) ;
16     tunda ( 1000 ) ;
17 }
```

sketsa.ino

diagram.json

Manajer Perpustakaan ▼

```
1  {
2      "versi" : 1 ,
3      "penulis" : "Anonim "
4      "editor" : "wokwi" ,
5      "bagian" : [
6          {
7              "tipe" : "board-esp32-devkit-c-v4" ,
8              "id" : "esp" ,
9              "atas" : 0 ,
10             "kiri" : 0 ,
11             "attrs" : { }
12         } ,
13         {
14             "tipe" : "dipimpin oleh wokwi" ,
15             "id" : "led1" ,
16             "atas" : -120 ,
17             "kiri" : 138,2 ,
18             "attrs" : { "color" : "red" }
19         } ,
20         {
21             "tipe" : "wokwi-resistor" ,
22             "id" : "r1" ,
23             "atas" : -35 ,
24             "kiri" : 105,6 ,
25             "attrs" : { "value" : "220" }
26         }
27     ] ,
28     "koneksi" : [
29         [ "esp:TX" , "$serialMonitor:RX" , "" , [ ] ] ,
30         [ "esp:RX" , "$serialMonitor:TX" , "" , [ ] ] ,
31         [ "esp:5" , "r1:1" , "hijau" , [ "h67.2" , "v-90" ] ] ,
32         [ "r1:2" , "led1:A" , "hijau" , [ "v-35" ] ] ,
33         [ "esp:GND.2" , "led1:C" , "hijau" , [ "h38.4" , "v-155" ] ]
34     ] ,
35     "serialMonitor" : { "tampilan" : "selalu" } ,
36     "ketergantungan" : { }
37 }
```

Pembahasan Latihan (Subbab 1.6)

Latihan 1: Pemahaman Logika Pin

- **Soal:** Mengapa menggunakan pin D8 (GPIO 15) dengan rangkaian *Active-High* (*pull-down*) dapat menyebabkan masalah saat *booting*?
- **Jawaban:** Pin D8 (GPIO 15) pada NodeMCU ESP8266 memiliki aturan khusus saat proses awal *booting*, yaitu kondisinya harus *LOW* (terhubung ke *Ground*). Jika kita memakai rangkaian *Active-High* dan menekan tombol saat alat baru dinyalakan, pin D8 akan memasukan tegangan 3.3V (*HIGH*). Akibatnya, chip ESP8266 bingung dan gagal membaca memori program (*boot failure*), sehingga program tidak berjalan dan alat menjadi *hang* sampai tombol dilepas lalu di-reset kembali.

Latihan 2: Implementasi Kode (Kedip LED Tidak Simetris)

- **Soal:** Modifikasi nilai `delay()` agar LED menyala 200 ms dan mati 800 ms.
- **Jawaban:** Untuk membuat jeda kedip tidak simetris, kita cukup mengubah angka pada fungsi `delay()` di dalam fungsi `loop()`. Berikut potongan kodenya:

C++

```
void loop() {  
    digitalWrite(ledPin, HIGH);  
    Serial.println("LED Menyala");  
    delay(200); // LED menyala sebentar (200 milidetik)  
  
    digitalWrite(ledPin, LOW);  
    Serial.println("LED Mati");  
    delay(800); // LED mati lebih lama (800 milidetik)  
}
```

Latihan 3: Analisis Sirkuit (Pemasangan LED Terbalik)

- **Soal:** Apa yang terjadi jika Anoda dan Katoda LED dipasang terbalik? Apakah ada *error* saat kompilasi?
- **Jawaban:** Jika kaki LED dipasang terbalik, **LED tidak akan menyala sama sekali**. Hal ini disebabkan LED adalah komponen dioda yang arusnya hanya bisa mengalir satu arah (Anoda ke Katoda). Pemasangan terbalik menahan aliran listrik yang masuk.

Meskipun LED tidak menyala, program **tidak akan mengalami error kompilasi** saat di-upload. *Compiler* hanya memeriksa apakah penulisan tata bahasa C++ sudah benar atau belum, dan tidak bisa mendeteksi kesalahan pemasangan kabel di fisik *breadboard*.

Latihan 4: Pengembangan Algoritma (Logika Terbalik)

- **Soal:** Ubah logika if-else agar saat tombol ditekan LED justru mati, dan saat dilepas LED menyala.
- **Jawaban:** Kondisi ini diselesaikan dengan membalik perintah digitalWrite pada blok if-else:

C++

```
void loop() {
    buttonState = digitalRead(buttonPin);

    if (buttonState == HIGH) {
        digitalWrite(ledPin, LOW); // Jika tombol ditekan, mati
        Serial.println("Tombol Ditekan -> LED MATI");
    } else {
        digitalWrite(ledPin, HIGH); // Jika tombol dilepas, nyala
        Serial.println("Tombol Dilepas -> LED MENYALA");
    }
}
```

Latihan Pembahasan Tugas Utama: Sistem Sakelar Toggle / Latching (Subbab 1.7)

Deskripsi Tugas: Membuat program tombol fisik agar berfungsi layaknya sakelar lampu ruangan. Tekan sekali lalu lepas maka LED akan menyala terus, tekan sekali lagi lalu lepas maka LED akan mati.

Kode Program C++ (tugas_prak01.ino):

C++

```
// Penentuan Pin

const int buttonPin = 4; // Pin D2 (GPIO 4)
const int ledPin = 5;    // Pin D1 (GPIO 5)

// Variabel Penjejak Status

bool statusLED = false;    // Menyimpan kondisi LED (false = OFF, true = ON)
int statusTombolLama = LOW; // Menyimpan status tombol pada pembacaan sebelumnya

void setup() {
    Serial.begin(115200);
```

```

pinMode(buttonPin, INPUT);
pinMode(ledPin, OUTPUT);

// Keadaan awal LED mati saat alat dinyalakan
digitalWrite(ledPin, LOW);
Serial.println("--- Sistem Sakelar Toggle Siap ---");
}

void loop() {
    int statusTombolSekarang = digitalRead(buttonPin);
    // Cek jika tombol baru saja ditekan (transisi dari LOW ke HIGH)
    if (statusTombolSekarang == HIGH && statusTombolLama == LOW) {
        // Balik status LED saat ini
        statusLED = !statusLED;

        // Terapkan perubahan ke LED fisik
        if (statusLED == true) {
            digitalWrite(ledPin, HIGH);
            Serial.println("Tombol Ditekan -> Sakelar ON (LED Menyala)");
        } else {
            digitalWrite(ledPin, LOW);
            Serial.println("Tombol Ditekan -> Sakelar OFF (LED Padam)");
        }
    }
    // Debouncing: Jeda 200 milidetik untuk meredam getaran mekanis tombol
    delay(200);
}

// Simpan nilai pembacaan untuk dicek pada loop berikutnya
statusTombolLama = statusTombolSekarang;
}

```

Penjelasan Cara Kerja Program:

1. **Penjejakan Status (*State Tracking*):** Variabel statusLED bertipe *boolean* digunakan sebagai memori untuk mengingat kondisi LED sedang *ON* atau *OFF*. Saat tombol ditekan, perintah statusLED = !statusLED akan membalik nilai tersebut secara otomatis.
2. **Peredam Getaran (*Debouncing*):** Penggunaan delay(200) berfungsi mencegah tombol membaca tekanan ganda akibat getaran mekanis plat seng di dalam tombol fisik saat ditekan, sehingga pergeseran status LED berjalan stabil.